# CSC 402: LANGUAGES AND COMPILERS

**Objectives:** To define what constitutes a language; classes of languages that are useful in describing computer languages, and how they can be mechanically recognized and how a high-level language (HLL) may be transformed into code (i.e. machine code) executable by a computer; lexical and syntax analyses of HLL's.

## CHAPTER1
## INTRODUCTION

### 1.1: Basic definitions

To study formal systems, we begin with some basic ideas based on the definitions below:

An **alphabet** is a finite non-empty set, $\sum$ of symbols. From these symbols, we construct **strings** which are finite ordered list of elements or sequences of symbols from the alphabet.

If the alphabet $\sum = \{2, 3, 5, 7,\}$ then 332 and 572 are strings on $\sum$.

With few exceptions, we will use lower case letters a, b, c, … for elements of and u, v, w, … for string names. Thus, if we write v = 72527, we mean the string named v has the specific value 72527. Repetitions can be specified with a superscript, for instance aabbbaccaaa = $a^2b^3ac^2a^3$ while $(ab)^3$ = ababab, etc

The **length** of a string s, |s| is the number of occurrences of symbols in s that is, its number of elements e.g. |v| =5.

An **empty string** is the string with no elements and is called the null string, denoted by λ which is a special string of length 0.

**Prefix/ Suffix**

A string obtained by removing zero or more tailing symbols of a string is a prefix e.g. if w = aeaae, the { λ, a, ae, aea, aeaa, aeaae} is the set of all prefixes of w while e, ae, aae are some of its suffixes.

The **Reverse** of a string is obtained by writing the symbols in reverse order; if t = 1243, then its reverse, denoted $t^R$ is 3421

**Substring**

Any string of consecutive characters in some w is said to be a substring of w. it is gotten by deleting a prefix or a suffix from w. e.g. eaa, aae, a etc are substrings of w.

Note that

1. every prefix and suffix of w is a substring of w but not all substrings can be a prefix or a suffix of w.
2. for every string w, w and λ are prefixes, suffixes and substrings of itself.

**Concatenation**

The concatenation of two strings u and v is the string obtained by appending the symbols of v to the right end of w, that is the string consisting of v followed by w, if w = 123 and v = abcd, then the concatenation of w and v, denoted by wv, is

wv = 123abcd and |wv| = |w| + |v|

The empty string λ is the identity element under concatenation, that is sλ = λs= s for any string, s.

**String Exponentiation**

If w is a string, we define the exponentiation of w as $w^n$ where

$$w^n = \begin{cases} \lambda, & n = 0 \\ w, & n = 1 \\ w^{n-1}w, & n > 1 \end{cases}$$

If $\sum$ is an alphabet, then we use $\sum^*$ to denote the set of all strings over $\sum$ obtained by concatenating zero or more symbols from $\sum$. E.g. let $\sum = \{a,b\}$, the $\sum^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$. The set $\sum^*$ always contains λ. To exclude the empty string, we define

$\sum^+ = \sum^* - \{\lambda\}$, ie all trings over $\sum$ except the null string.

While $\sum$ is finite, $\sum^+$ and $\sum^*$ are always infinite since there is no limit on the length of the strings in these sets. With all these definitions seen above, we can now define a language.

### $1.2: Language

Informally, **a language**, is a system suitable for the expression of certain idea, facts, or concepts, including a set of symbols and rule for their manipulation.

**A language** is defined very generally as a subset of $\sum^*$. Thus it is a set of strings over some fixed alphabet $\sum$. By this definition, $\{\lambda\}$ is a language. A string in a language L is called a **sentence** of L.
E.g. if $\sum = \{a,b\}$. Then the set {a, aa, aab} is a language on $\sum$. It is a finite language because it has a finite number of sentences. Considering the set $L = \{a^n b^n : n>0\}$ is also a language on $\sum$. The strings aabb and aaaabbbb are in L but the string abb, aab are not sentences in L. This language is infinite. Most interesting languages are infinite.

### 1.2.1: Operations on languages.

The following operations can be performed on languages. Let L and M be languages, then

1. Union. $L \cup M = \{ s | s \text{ is in L or s is in M}\}$
2. Intersection. $L \cap M = \{ s | s \text{ is in L and s is in M}\}$
3. Concatenation. $LM = \{st | s \text{ is in L and t is in M}\}$
4. The difference. $L-M = \{ s | s \text{ is in L and s is not in M}\}$
5. The complement of L is $L^C = \sum^* - L$.
6. The reverse of a language is the set of all string reversals, that is $L^R = \{ w^R : w \in L\}$.
7. Exponentiation (Concatenation with itself, n times).
8. Kleene closure or star closure. $L^* = L^0 \cup L^1 \cup L^2 \ldots$ zero or more concatenation of L with itself.
9. Positive Closure. $L+ = L^1 \cup L^2 \ldots$. One or more concatenation of L with itself.

Examples:
1.If $L = \{a^n b^n : n \geq 0\}$, then
$L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}$.
Note that n and m above are unrelated; the string aabbaaabbb is in $L^2$.
The reverse of L is easily described in set notations as
$L^R = \{b^n a^n : n \geq 0\}$

2. If $L = \{0, 01\}$, then $L^2 = LL = \{00, 001, 010, 0101\}$ and
$L^* = L^0 \cup L^1 \cup L^2 \cup \ldots\ldots\ldots$
$= \{\lambda\} \cup \{0,01\} \cup \{00,001,010, 0101\} \ldots\ldots$ comprises all binary strings in which every 1 is preceded by a 0.
$L^+ = L^1 \cup L^2 \cup L^3 \cup \ldots\ldots\ldots$
$= \{0,01\} \cup \{00,001,010, 0101\} \ldots\ldots$

Note:
1. $L+ = LL^*$
2. $-\lambda$ is in $L^*$ and $\lambda$ is in $L+$ if and only if $\lambda$ is in L.
3. $\sum^*$ is the kleene closure of $\sum$ when $\sum$ is viewed as a language of words of length 1 and $\sum^+$ is just the positive closure of $\sum$.

$L_1 = \{01, 11, 0110\}$ and $L_2 = \{0^n 1^n | n \geq 0\}$ are two languages over the binary alphabet {0, 1}. $L_1$ has three words, while $L_2$ is infinite. The string 01 is in both languages while 11 is in $L_1$ but not in $L_2$.

### 1.2.2: Representation of languages.

In general, a language over an alphabet $\sum$ is a subset of $\sum^*$. How can we describe a language rigorously so that we know whether a given string belongs to the language or not? We have seen that finite languages such as $L_1$ can be explicitly defined by enumerating its elements. An infinite language such as $L_2$ cannot be exhaustively enumerated but in some cases we are able to a simple rule characterizing all of its members. The question now is "can we define systematic methods for defining rules that characterize a wide class of languages? We shall address this issue as we progress in this section of the course.

## 1.3: General Grammars.

One of the most useful and general system for representing languages is based on the formal notion of a grammar.

**1.3.1 Definition**: A grammar is a quadruple $(\sum, V, S, P)$, where:

a. $\sum$ is a finite non-empty set called the terminal alphabet. The elements of $\sum$ are called the terminals.
b. V is a finite non-empty set, disjoint from $\sum$. The elements of V are called the non-terminals or variables.
c. $S \in V$ is a distinguished non-terminal called the start symbol.
d. P is a finite set of productions (or rules) of the form $\alpha \rightarrow \beta$ where $\alpha \in (\sum \cup V)^* V (\sum \cup V)^*$ and $\beta \in (\sum \cup V)^*$, ie $\alpha$ is a string of terminals and non-terminals containing at least one non-terminal and $\beta$ is a string of terminals and non-terminals.

E.g.

a. Let $G_1 = (\{0, 1\}, \{S, T, O, I\}, S, P)$ where P contains the following productions:

$S \rightarrow OT$
$S \rightarrow OI$
$T \rightarrow SI$
$O \rightarrow 0$
$I \rightarrow 1.$

As we shall see, the grammar $G_1$ can be used to describe the set $\{0^n1^n | n \geq 1\}$.

b. Let $G2 = (\{0, 1, 2\}, \{S, A\}, S, P)$ where P contains the following productions:

$S \rightarrow 0SA2$
$S \rightarrow \lambda$
$2A \rightarrow A2$
$0A \rightarrow 01$
$1A \rightarrow 11.$

This grammar $G_2$ can be used to describe the set $\{0^n1^n2^n \mid n \geq 0\}$.

**1.3.2 Definitions**: Let $(\sum, V, S, P)$ be a grammar.

a. A sentential form of G is any string of terminals and non-terminals ie a string over $\sum \cup V$.
b. Suppose $w_1$, $w_2$ are two sentential forms of G. We say that w1 directly derives $w_2$, written $w_1 \Rightarrow w_2$ if $w_1 = \delta\alpha\tau$, $w_2 = \delta\beta\tau$ and $\alpha \rightarrow \beta$ is a production in P.

For example, the sentential form 00S11 directly derives the sentential form 00OT11 in grammar $G_1$ and A2A2 directly derives AA22 in grammar $G_2$.

c. Suppose $w_1$ and $w_2$ are two sentential forms of a grammar G. We say that $w_1$ derives $w_2$ written $w_1 \Rightarrow^* w_2$ if there exists a sequence of (zero or more) sentential forms $\delta_1, \ldots, \delta_n$ such that $w_1 \Rightarrow \delta_1 \Rightarrow \ldots \Rightarrow \delta_n \Rightarrow w_2$.
d. The sequence $w_1 \Rightarrow \delta_1 \Rightarrow \ldots \Rightarrow \delta_n \Rightarrow w_2$ is called a derivation of $w_2$ from $w_1$.

For example, in grammar $G_1$, $S \Rightarrow^* 0011$ because
$S \Rightarrow OT \Rightarrow 0T \Rightarrow 0SI \Rightarrow 0S1 \Rightarrow 0OI1 \Rightarrow 00I1 \Rightarrow 0011$ and in grammar $G_2$, $S \Rightarrow^* 001122$ because
$S \Rightarrow 0SA2 \Rightarrow 00SA2A2 \Rightarrow 00A2A2 \Rightarrow 0012A2 \Rightarrow 0011A22 \Rightarrow oo1122.$

**1.3.3 Definition**: Let $(\sum, V, S, P)$ be a grammar. The language generated by G, denoted L(G), is defined as
$L(G) = \{x \mid x \in S \Rightarrow^* x\}$.

The words in L(G) are also called sentence of L(G).

Clearly, $L(G_1) = \{0^n1^n \mid n \geq 1\}$ and $L(G_2) = \{0^n1^n2^n \mid n \geq 0\}$.

We now introduce a shorter notation in which several production rules with the same left-hand side are written on a single line with alternative right-hand sides separated by |. In this notation, the productions

$S \rightarrow aAb$ and $S \rightarrow \lambda$ are written as $S \rightarrow aAb \mid \lambda$.

**1.3.4 Example**.

1. The set of all typical legal identifiers in Pascal is a language. Informally, it is the set of all strings starting with a letter and followed by an arbitrary number of letters or digits. The grammar below makes this informal definition precise.

({a,b,…,z,0,1,…,9}, {<id>, <rest>, digit>, <letter>,<id>, P) with productions as follows:

<id> → <letter><rest>

<rest> → <letter><rest>|<digit><rest>|λ

<letter> → a|b|…|z

<digit> → 0|1|…|9.

Derive the identifier a0.

**1.4: Automata**.

Automata: an algorithm or program that automatically recognizes if a particular string belongs to the language or not, by checking the grammar of the string.

An automaton is an abstract computing device (model of computation or machine) of a digital computer. There are different varieties of such machines which can be defined mathematically.

Every automaton fulfills three basic requirements.

As such includes some essential features:

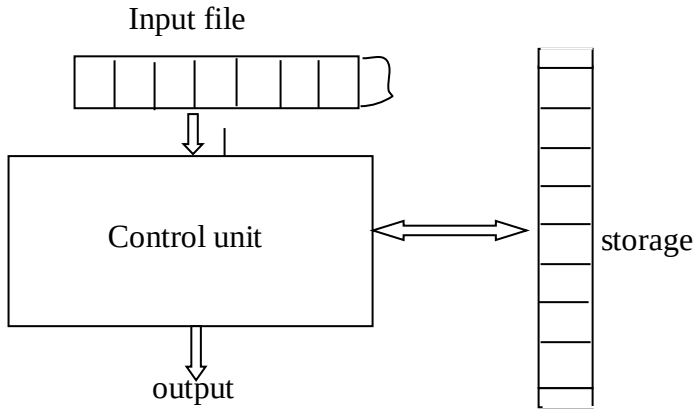
-**A mechanism for reading input**. It will be assumed that the input is a string over a given alphabet written on an input file or tape which the automaton can read but not change. This input file is divided into cells, each of which can hold one symbol. This input mechanism reads the input file from left to right, one symbol at a time and detects the end of the input string by sensing an end-of-file condition. The simpler automata can only read the input one symbol at a time from left to right but not change. Powerful versions can both read (from left to right or right to left) and change the input.

-**The automaton can produce output of some form**. If the output in response to an input string is binary (say, accept or reject), then it is called an accepter. If it produces an output sequence in response to an input sequence, then it is called a transducer(or automaton with output).

-**It may have a temporary** storage device consisting of an unlimited number of cells, each capable of holding a single symbol from an alphabet (not necessarily the same one as the input alphabet). The automaton can read and change the contents of the storage cells.

-The automaton has a control unit which can be in any of a finite number of internal states and which can change state in some defined manner determined by a transition function.

**Schematic representation of a general automaton.**




An automaton is assumed to operate in a discrete time frame. At any given time, the control unit is in some internal state, and the input mechanism is scanning a particular symbol in the input file. The internal state of thee control unit at the next time step is determined by the next-state or transition function. This

transition function gives the next state in terms of the current state, the current input symbol, and the information currently in the temporary storage. During the transition from one time interval to the next, output may be produced or the information in the temporary storage changed. The term configuration will be used to refer to a particular state of the control unit, input file , and temporary storage. The transition of the automaton from one configuration to the next will be called a move.

An automaton whose output response is limited to a simple 'yes' or 'no' is called an accepter. Presented with an input string, an accepter either accepts the string or rejects it. A more general automaton, capable of producing strings of symbols as output is called a transducer.

An automaton can be represented by a graph in which the vertices give the internal states and the edges, transitions. The labels on the edges show what happens (in terms of input and output) during the transition.

For example

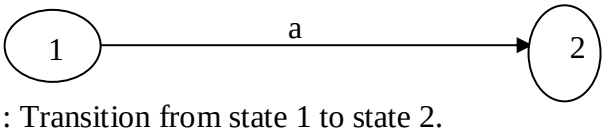
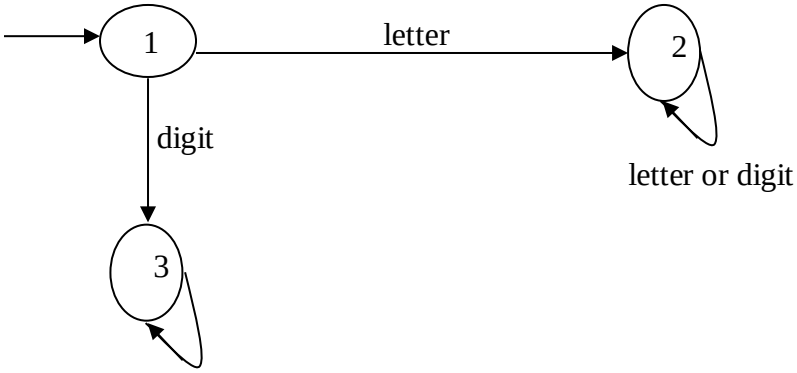
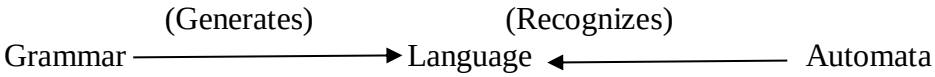



Figure : Transition from state 1 to state 2.
This figure represents a transition from state1 to state2, which is taken when the input symbol is a.

E.G . An automaton(transition graph) describing all legal Pascal identifiers is as follows:




We assume that initially, the automaton is in state1 by an arrow not originating in any vertex to this state. When the first symbol (read from left to right as usual) is not a letter, the automaton goes into state2, after which the rest of the string is immaterial. State2 therefore represents a 'yes' state of the accepter. Conversely, if the first symbol is a digit, the automaton will go into state3, the 'no' state and remain there. Here, we assume that no input other than letters or digits is possible.




# CHAPTER 2
# REGULAR LANGUAGES

## 2.1 FINITE AUTOMATA

**States, Transitions and Finite-State Transition System :**

Informally,
**a state of a system** is an instantaneous description of that system which gives all relevant information necessary to determine how the system can evolve from that point on.
**Transitions** are changes of states that can occur spontaneously or in response to inputs to the states. Though transitions usually take time, we assume that state transitions are instantaneous (which is an abstraction).
Some examples of state transition systems are: digital systems, vending machines, etc. A system containing only a finite number of states and transitions among them is called a finite-state transition system. Finite-state transition systems can be modeled abstractly by a mathematical model called finite automation
A finite accepter is an automaton characterized by having no temporary storage. Since an input cannot be written, a finite automaton is severely limited in its capacity to 'remember' things during the computation. A finite amount of information can be retained in the control unit by placing the unit into a specific state. A finite automaton can only deal with situations in which the information to be stored at any time is strictly bounded since the number of states is finite. It is actually a finite state machine but has no output and a set of states called accepting or final states. We now have the following:

### 2.1.1 Deterministic Finite accepters (dfa)

Informally, a DFA (Deterministic Finite State Automaton) is a simple machine that reads an input string --one symbol at a time -- and then, after the input has been completely read, decides whether to accept or reject the input. As the symbols are read from the tape, the automaton can change its state, to reflect how it reacts to what it has seen so far. A machine for which a deterministic code can be formulated, and if there is only one unique way to formulate the code, then the machine is called deterministic finite automata. Thus, a DFA conceptually consists of 3 parts:

1. A *tape* to hold the input string. The tape is divided into a finite number of cells. Each cell holds a symbol from an qlphabet $\sum$.
2. A *tape head* for reading symbols from the tape
3. A *control* , which itself consists of 3 things:
   i. finite number of states that the machine is allowed to be in (zero or more states are designated as *accept* or *final* states),
   ii. a current state, initially set to a start state,
   iii. a state transition function for changing the current state.

An automaton processes a string on the tape by repeating the following actions until the tape head has traversed the entire string:

1. The tape head reads the current tape cell and sends the symbol s found there to the control. Then the tape head moves to the next cell.
2. he control takes s and the current state and consults the state transition function to get the next state, which becomes the new current state.

Once the entire string has been processed, the state in which the automation enters is examined.

If it is an accept state , the input string is accepted ; otherwise, the string is rejected . Summarizing all the above we can formulate the following formal definition:

**2.1.1.1: Definition**

A deterministic finite accepter or dfa is defined by the quintuple

$M = (Q, \sum, \delta, q_o, F)$ where

Q is a finite set of internal states

$\sum$ is a finite set of symbols called the input alphabet

$\delta$: Q x $\sum \rightarrow$ Q is a total function called the transition function,

$q_o \in Q$ is the initial state

$F \subseteq Q$ is a set of final states.

At the initial time, the dfa is assumed to be in the initial state $q_o$ with its input mechanism on the leftmost symbol of the string. During each move, the input mechanism advances one position to the right, so each move consumes one and only one input symbol. When the end of the string is reached, the string is accepted if the automaton is in one of its final states, otherwise the string is rejected. The transitions from one internal state to another are governed by the transition function $\delta$. E.g. if $\delta(q_o, a) = q_1$, then if the dfa is in state $q_o$ and the current input symbol is a, the dfa will go into state $q_1$.

**Transition table:**

It is basically a tabular representation of the transition function that takes two arguments (a state and a symbol) and returns a value (the "next state").

- Rows correspond to states,
- Columns correspond to input symbols,
- Entries correspond to next states
- The start state is marked with an arrow
- The accept states are marked with a star (*)

**(State) Transition diagram:**

A state transition diagram or simply a transition diagram is a directed graph which can be constructed as follows:

1. For each state Q there is a node.
2. There is a directed edge from node q to node p labelled a if and only if $\delta(q_o, a) = p$. if there are several input symbols that cause a transition, the edge is labelled by the list of these symbols.
3. There is an arrow with no source into the start state.
4. Accepting states are indicated by double circle.

We use transition graphs to visualize and represent finite automata in which the vertices represent transitions. The labels on the vertices are the names of the states, while the labels on the edges are the current values of the input symbol. The initial state will be identified by an incoming unlabelled arrow at originating at any vertex. An edge $(q_o, q_1)$ labelled **a** represents the transition $\delta(q_o, a) = q_1$.

Thus, if $M = (Q, \sum, \delta, q_o, F)$ is a deterministic finite accepter, then its associated transition graph $G_M$ has exactly |Q| vertices each labelled with a different $q_i \in Q$.

**2.1.1.2: Definition**

The language accepted by a dfa $M = (Q, \sum, \delta, q_o, F)$ is the set of all strings on $\sum$ accepted by M. In formal notation, $L(M)=\{w \in \sum^* : \delta^*(q_o,w) \in F\}$. Where $\delta^*$ is called the extended function and it gives the state the automaton reaches when it starts from the state $q_0$ and finish processing the string w.

Note. Non-acceptance of a string means that the dfa stops in a non-final state. Write the complement of L(M) in set notation.

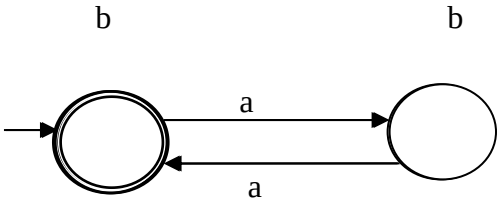
**2.1.1.3: Examples**

1. The following transition graphs accepts any string made up of on {a, b}

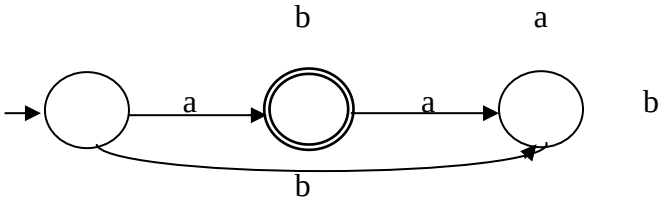
a. a's and b's ending with an a.



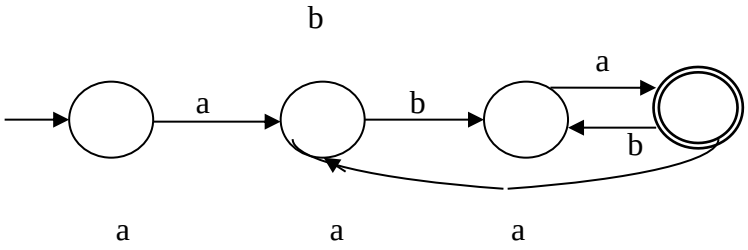

b. a's and b's with exactly an even number of a's



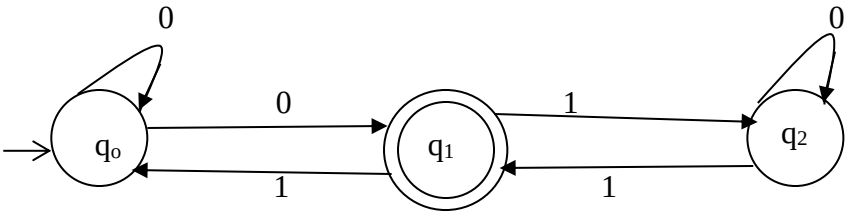

c. starting with one a followed by any number of b's




d.

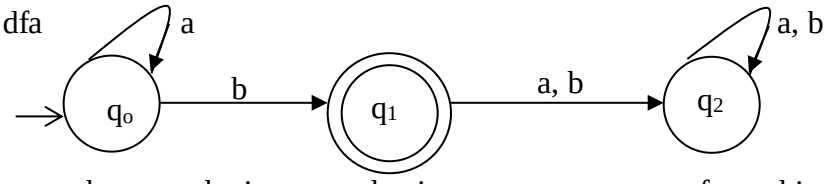



2. The graph




Represents the dfa M = ({$q_o$, $q_1$, $q_2$}, {0, 1}, δ, qo, {q1}), where δ is given by

$\delta(q_o, 0) = q_o$, $\delta(q_o, 1) = q_1$
$\delta(q_1, 0) = q_o$, $\delta(q_1, 1) = q_2$
$\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$.
This dfa accepts the string 01. Starting in state $q_o$, the symbol 0 is read first. This dfa does not accept the string 00 since after reading consecutive 0's, it will be in state $q_o$. By similar reasoning, we see that the automaton will accept the strings 101, 0111, and 11001, but not 100 or 1100.

3. Consider the dfa 

State $q_2$ is a trap state because the input mechanism can never escape from this state once it is there. We can verify that in set notation, the language accepted by the automaton is $L= \{a^n b: n \geq 0\}$.

3. Find a deterministic finite accepter that recognizes the set of all strings on $\sum = \{a, b\}$ starting with the prefix ab.
4. Design a dfa on the alphabet {0, 1} having atleast one 1

5. Find a dfa that accepts all the strings on {0, 1}, except those containing the substring 001.
Try some strings.

### 2.1.2 Nondeterministic Finite Accepters (ndfa)

Finite accepters are more complicated if we allow them to act non-deterministically. Non-determinism is a powerful, but at first sight, unusual idea. We normally think of computers as completely deterministic, and the element of choice seems out of place. Nevertheless, non-determinism is a useful notion as we shall see.

Non-determinism means a choice of moves for an automaton. Rather than prescribing a unique move in each situation, we allow a set of possible moves. Formally, we achieve this by defining the transition function so that its range is a set of possible states.

It is easy to construct and comprehend an NFA than DFA for a given regular language. The concept of NFA can also be used in proving many theorems and results. Hence, it plays an important role in this subject. In the context of FA non-determinism can be incorporated naturally. That is, an NFA is defined in the same way as the DFA but with the following two exceptions:
□□multiple next state.
- transitions.

**Multiple Next State :**

□□In contrast to a DFA, the next state is not necessarily uniquely determined by the current state and input symbol in case of an NFA. (Recall that, in a DFA there is exactly one start state and exactly one transition out of every state for each symbol in $\sum$).

□□This means that - in a state *q* and with input symbol a - there could be one, more than one or zero next state to go, i.e. the value of $\delta(q, a)$ is a subset of *Q*. Thus $\delta(q, 0) = \{q, r, s, t \ldots\ldots l\}$ which means that any one of could be the next state.

- The zero next state case is a special one giving $\delta(q_o, 0) = \emptyset$ , which means that there is no next state on input symbol when the automata is in state *q*. In such a case, we may think that the automata "hangs" and the input will be rejected.

**λ- transitions :**

In a λ-transition, the tape head doesn't do anything- it does not read and it does not move. However, the state of the automata can be changed - that is can go to zero, one or more states. This is written formally $\delta(q, \lambda) = \{q, ....\}$ as implying that the next state could by any one of {q, ….} without consuming the next input symbol.

**Acceptance :**

**2.1.2.1: Definition**

A non-deterministic finite accepter or nfa is defined by the quintuple $M = (Q, \sum, \delta, q_0, F)$, where Q, ∑, $q_0$, F are defined as for dfa's but $\delta$: Q x $(\sum \cup \{\lambda\}) \rightarrow P(Q)$.

Note that there are three major differences between the definition of nfa and dfa.

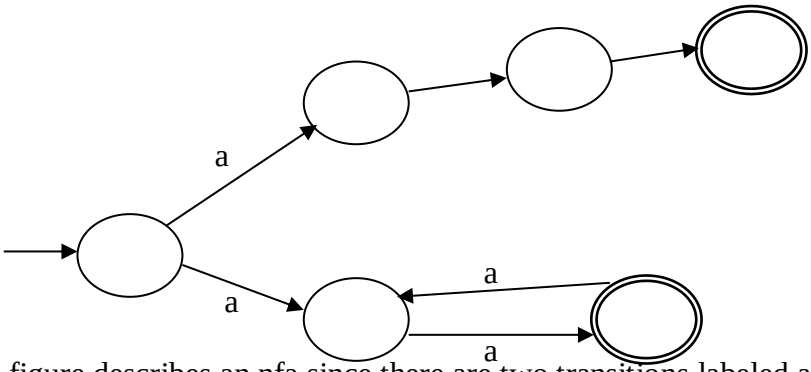
i) In nfa, the range of λ is in the powerset of Q, so that its value is not a single element of Q but a subset of it.

ii) λ is allowed as a second argument of δ that is , the nfa can make a move without consuming an input, although we still assume that the input mechanism can only travel to the right, it is possible that it is stationary on some moves.

iii) In an nfa, the set $\lambda(q_i, a)$ may be empty, meaning that there is no transition defined for this specific situation. So some edges may be labelled λ.

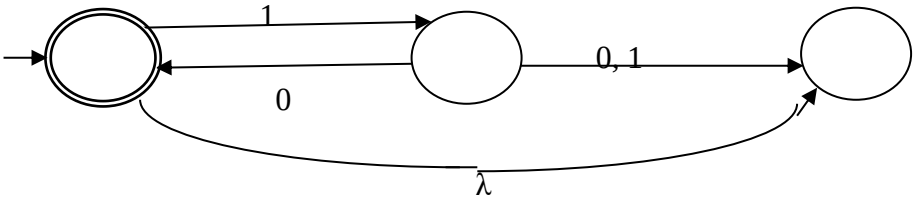
**2.1.2.2: Examples**.

1.



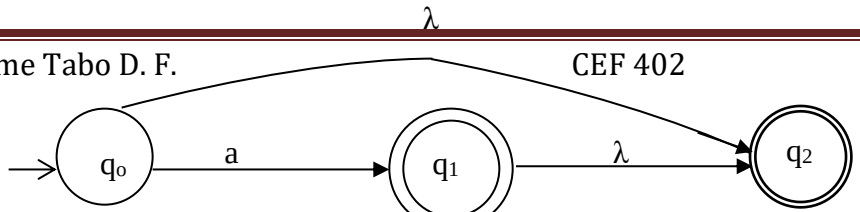

This figure describes an nfa since there are two transitions labeled a out of the initial state.

2. A non-deterministic automaton is shown below. It is non-deterministic not only because several edges with the same label originate from one vertex, but because it has an l-transition. Some transitions such as $\delta(q_2,0)$ are unspecified in the graph. This is to be interpreted as a transition to the empty set ie $\delta(q_2,0) = \varnothing$.




This automaton accepts strings such as λ, 1010, and 101010 but not 110 and 10100. For 10, there are two alternative walks: one leading to $q_0$, the final state and the other to $q_2$. Even though $q_2$ is not a final state, the string is accepted because one walk leads to a final state.





3. Consider the nfa

Find $\delta^*(q_1, a)$
$\delta^*(q_2, \lambda)$
$\delta^*(q_2, aa)$.

4. What is the language accepted by the automaton in example two above. It is easy to see that it is $L = \{(10)^n: n \geq 0\}$. Test the strings.
Testing 110, we see that after reading 11, the automaton finds itself in state q1 with $\delta(q_2, 0)$ undefined. We call such a situation a dead configuration, and we visualize it as the automaton simply stopping without further action. Precisely, we can say that $\delta^*(q_0, 110) = \varnothing$. Thus no final state can be reached by processing 110 hence the string is not accepted.

**2.1.2.3: Definition**
The language L accepted by an ndfa $M = (Q, \sum, \lambda, qo, F)$ is defined as the set of all strings accepted in the above sense.
Formally, $L(M) = \{w \in \sum^*: \delta^*(q_0, w) \cap F \neq \varnothing\}$.
In other words, the language consists of all strings w for which there is a walk labeled w from the initial vertex of the transition graph to some final vertex.
What is the language accepted by the automaton in **example 2.1.2.2(2)**?

## 2.2: Regular Expressions

**2.2.1 Definition** Regular expressions are constructed from primitive constituents (basic elements) by repeatedly applying certain recursive rules. That is
Let $\sum$ be a given alphabet. Then regular expressions are defined recursively as follows
1. $\varnothing$, $\lambda$, and $a \in \sum$ are all regular expressions. These are called primitive regular expressions.
2. If $r_1$ and $r_2$ are regular expressions, so are $r_1+r_2$, $r_1.r_2$, $(r_1)$ and $r_1^*$.
3. A string is a regular expression if and only if it can be derived from the primitive regular expressions by a finite number of applications of the rules in (2).

For example $\sum = \{a, b, c\}$, the string $(a + b.c)^*.(c + \varnothing)$ is a regular expression since it is constructed by application of the above rules. Note (a+b.c)* stands for
$(\{a\} \cup \{bc\})^* = \{\lambda, a, bc, aa, abc, bca, bcbc, aaa, aabc, \ldots\}$
Regular languages can be described using regular expressions. If r is a regular expression, we will let L(r) denote the language associated with r. that is the language L(r) denoted by any regular expression r is defined by the following rules.
- $\varnothing$ is a regular expression denoting the language { },
- $\lambda$ is a regular expression denoting $\{\lambda\}$,
- For every $a \in \sum$, a is a regular expression representing the language {a},
- Suppose r and s are regular languages representing the languages R and S, then (r + s), (rs), (r)* are regular expressions that represent the languages $R \cup S$, RS and R* respectively.

The specification of a regular expression is an example of a recursive definition. Rules 1 to 3 form the basis of the definition while Rule 4 provides the inductive step.

Since the set operations: union and concatenation are both associative, and since we can stipulate that Kleene closure takes precedence over concatenation and concatenation over union, many parentheses can be omitted from regular expressions.

**Examples**.

**1**. ((0(0+1)*)+((0+1)*0)) is a regular expression over {0, 1} that represents the language consisting of all binary strings that begin or end with a 0. This can be written as 0(0+1)*+(0+1)*0.

**2**. The expression 0(0+1)* represents the set of all srtings that begin with a 0 and end with a 1.

- 

If $r_1$ and $r_2$ are regular expressions, then

- $L(r_1+r_2) = L(r_1) \cup L(r_2)$,
- $L(r_1.r_2) = L(r_1)L(r_2)$,
- $L((r_1)) = L(r_1)$,
- $L(r_1^*) = (L(r_1))^*$.

We apply rules 4) to 7) to reduce L(r) to simpler components recursively and to see what language a given expression denotes. The first three are the termination conditions for this recursion.

**3.4: Examples**.

1. List some elements belonging to L(a*.(a+b)).

L(a*.(a+b)) = L(a*)L(a+b)

= (L(a))*(L(a) ∪L(b))

= {λ, a, aa, aaa, …}{a, b}

= {a, aa, aaa, aaaa, …, b, ab, aab, aaab, …}.

Here, star-closure precedes concatenation and concatenation precedes union. Also, $r_1r_2$ can be used for $r_1.r_2$. So that L(a.b+c) = {ab, c} instead of L(a.b+c) = {ab, ac}

2. For ∑ = {a, b}, the expression r = (a + b)*(a + bb) is regular. It denotes the language L® = {a, bb, aa, abb, ba, …}. We can see this by considering the various parts of r. The first part (a+b)* stands for any string of a's and b's. the second part, (a+bb) represents either an a or a double b. consequently, L(r) is the set of all strings on {a, b}, terminated by either an a or a bb.

3. The expression r = (aa)*(bb)*b

Denotes the set of all strings with an even number of a's followed by an odd number of b's that is L(r) = $\{a^{2n}b^{2m+1}: n \geq 0, m \geq 0\}$.

4. For ∑ = {0, 1}, give a regular expression r such that

L(r) = {w∈∑*: w has at least one pair of consecutive zeros}

Here, one can arrive at an answer by reasoning something like this: Every string in L(r) must contain 00 somewhere, but what comes before and after is anystring. An arbitrary string on {0, 1} can be denoted by (0+1)*. Putting these observations together, we arrive at the solution as

r = (0+1)*00(0+1)*.

5. Find a regular expression for the language

L = {w∈{0, 1}*: w has no pair of consecutive zeros}.

**2.3 Regular grammars**.

We now look at grammars that generate regular languages.

**2.3.1: Definitions.**

**1.** A grammar G = (∑, V, S, P) is said to be right-linear if all productions are of the form

$A \rightarrow xB$,

$A \rightarrow x$ where A, B ∈ V and x ∈ ∑*.

**2.** A grammar is said to be left-linear if all productions are of the form

$A \rightarrow Bx$,

$A \rightarrow x$ where A, B ∈ V and x ∈ ∑*.

**3**. A linear grammar is a grammar in which at most one variable can occur on the right side of any production without restriction on the position of this variable.

**4.** A regular grammar is one that is either right-linear or left-linear.

**Note:**

Clearly, a regular grammar is always linear, but not all linear grammars are regular. In a regular grammar, at most one variable appears on the right side of any production. Furthermore, that variable must consistently be either the rightmost or leftmost symbol of the right side of any production.

**2.3.2: Examples**.

**1.** $G_1 = (\{a, b\}, \{S\}, S, P_1)$ with P1 given as

$S \longrightarrow abS \mid a$

is right-linear.

$G_2 = (\{a, b\}, \{S, S_1, S_2\}, S, P_2)$ with productions

$S \rightarrow S_1ab$,

$S_1 \rightarrow S_1ab \mid S_2$,

$S_2 \rightarrow a$

is left-linear. Both $G_1$ and $G_2$ are regular grammars.

**2**. The grammar G = ({S, A, B}, {a, b}, S, P) with productions

$S \rightarrow A$,

$A \rightarrow aB \mid \lambda$,

$B \rightarrow Ab$

is not regular.

Although every production is either in right-linear or left-linear form, the grammar itself is neither right- linear nor left-linear and therefore is not regular. This grammar is an example of a linear grammar.

**2.3: Equivalence of dfa's and ndfa's**.

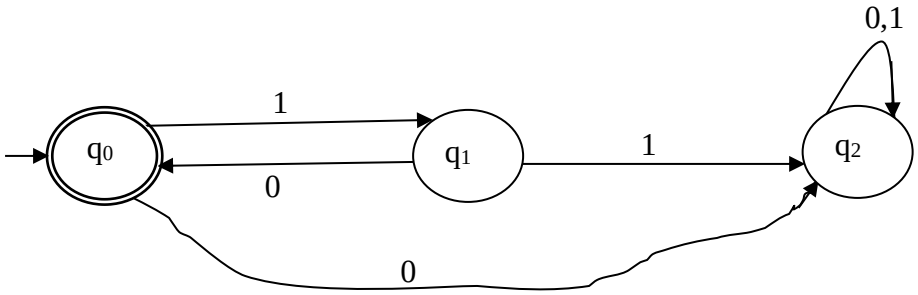
Obviously, there is a difference in the definitions of dfa's and ndfa's, but this does not imply that there is any essential distinction between them.

It is worth noting that a *DFA* is a special type of *NFA* and hence the class of languages accepted by *DFA* s is a subset of the class of languages accepted by *NFA* s. Surprisingly, these two classes are in fact equal. *NFA* s appeared to have more power than *DFA* s because of generality enjoyed in terms of λ-transition and multiple next states. But they are no more powerful than *DFA* s in terms of the languages they accept.

**2.3.1: Definition**.
Two finite accepters $M_1$ and $M_2$ are said to be equivalent if $L(M_1) = L(M_2)$, that is, if they both accept the same language.

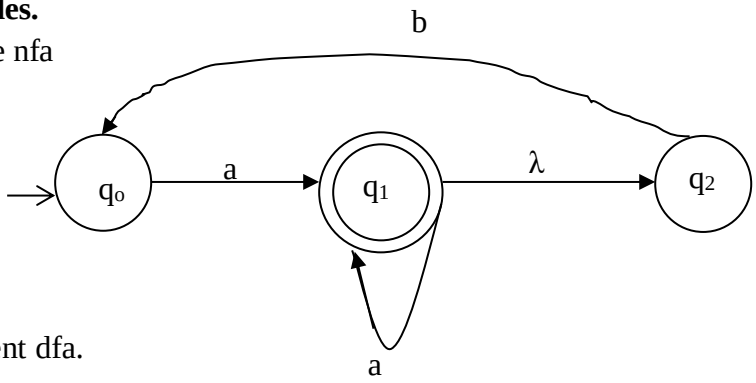
Note as mentioned before, there are generally many accepters for a given language, so any dfa or ndfa has many equivalent accepters.

**2.3.2: Examples.**
The dfa below and the nfa in **example 2.2.2(2)** are equivalent and both accept the language $\{(10)^n: n \geq 0.\}$



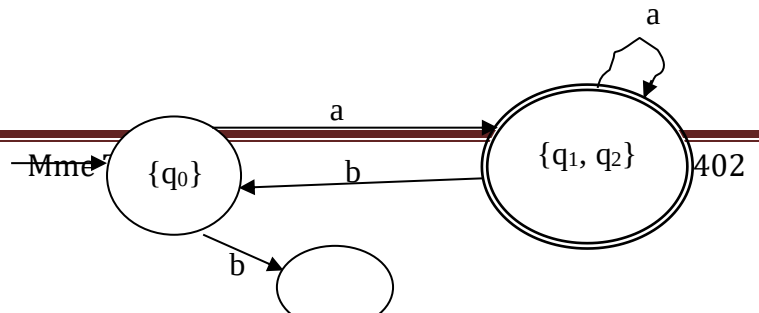

Note also that the classes of dfa's and ndfa's are equally powerful. It can be shown that for every language accepted by some nfa, there is a dfa that accepts the same language.

**2.3.3: examples.**
1. Convert the nfa




to an equivalent dfa.

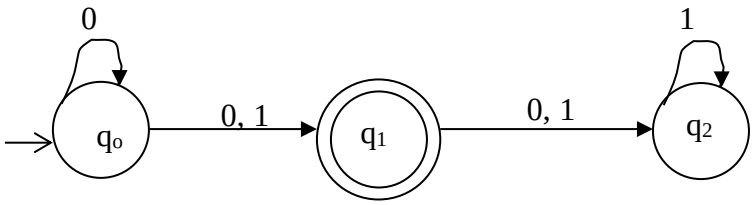
The ndfa starts in state $q_0$, so the initial state of the dfa will be labeled {q0}. After reading an a, the nfa can be in state $q_1$ or making a λ-transition, in state $q_2$. Therefore, the corresponding dfa must have a state labeled $\{q_1, q_2\}$ and a transition $\delta(\{q_0\}, a) = \{q_1, q_2\}$.
A state labeled ∅ represents an impossible move for the nfa, therefore means non-acceptance of the string. Consequently, this state in the dfa must be a non-final trap state. Now we have introduced the state $[q_1, q_2]$, so we need to find the transitions out of this state. Remember that this state of the dfa corresponds to two possible states of the nfa. So we must refer back to the nfa. If the nfa is in state $q_1$ and reads an a, it can go to $q_1$. Furthermore, from $q_1$ the nfa can make a λ- transition to $q_2$. If for the same input, the nfa is in state $q_2$, then there is no specified transition.
Therefore, $\delta(\{q_1, q_2\}, a) = \{q_1, q_2\}$.
Similarly, $\delta\{q_1, q_2\}, b) = \{q_0\}$.
At this point, we have the dfa as



{ $q_1, q_2$}

a, b

2. Convert




into an equivalent deterministic machine.

Theorem: Every DFA has as equivalent NFA and equivalently, given any NFA, there is an equivalent DFA
The formal construction of an equivalent DFA for any NFA is given as follows.
We first consider an NFA without λ-transitions and then we incorporate the effects of λ-transitions later.

Given an NFA, N = (Q, ∑, $\delta$, q0, F) without λ-transitions, we construct an equivalent DFA, D = ($Q_D$, ∑, $\delta_D$, $q_{0D}$, $F_D$) as follows

$Q_D = P(Q)$

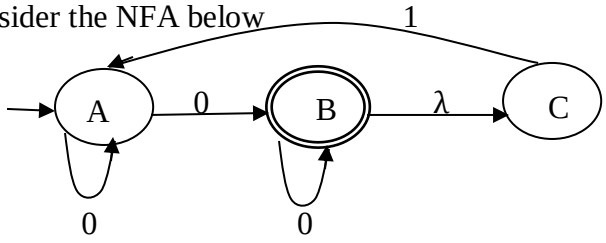
$q_{0D} = \{q_0\}$.

$F_D = \{q_D \in Q_D \mid q_D \cap F \neq \emptyset$

$\delta_D(\{q_1,q_2, ..., q_k\},a) = \delta(q_1,a) \cup \delta(q_2, a) \cup ... \cup \delta(q_k,a)$ for all a ∈∑ and $Q_D = \{q_1,q_2, ..., q_k\}$, where qi ∈ Q, $1 \leq i \leq k$

Example

Consider the NFA below




Convert it to a DFA.

**2.4: Minimisation of the Number of States in a Finite Automata**

Most often, DFAs constructed NFAs are not minimal though they are rarely far from being minimal eg the DFA at the beginning of this page has two states that are equivalent: neither are accepting and they have identical transitions, it is not minimal eventhough it has only four states. DFAs can be minimized by collapsing equivalent states. At the beginning, we assume all states are identical and then separate them if we can prove them different. The following rules are used>

- An accepting state is not equivalent to a non-accepting state.
- If two states $q_i$ and $q_t$ have transitions on the same symbol, a, to states $q_j$ and $q_s$ that we have already proven to be different, then qi or qt have a defined transition on a.
-

**Algorithm 2.4 (Minimisation of the Number of States in a Finite Automata):** Given a DFA D over an alphabet $\sum$ with states Q and $F \subseteq Q$ are the accepting states, we construct a minimal DFA D' where each state is a group of states from D. the groups in the minimal DFA are consistent: For any pair of states $q_l$ and $q_k$, in the same group G and any symbol a, move (ql, a) is in the same group G' as move ($q_k$, a) or both are undefined.

i. We start with two groups: F and Q\F. these are unmarked.
ii. We pick any unmarked group G and check if it is consistent. If it is, we mark it. If G is not consistent, we split it into maximal consistent subgroups and replace G by these. All groups are then unmarked.
iii. If there are no unmarked groups left, we are done and the remaining groups are the states of the minimal DFA. Otherwise we go back to step 2.

The starting state of the minimal DFA is the group that contains the original starting state and any group of accepting states is an accepting state in the minimal DFA.

**2.4.1Example:** The transition graph in **figure 1** below can be minimized to that in **figure 2** below.

**Figure 1**

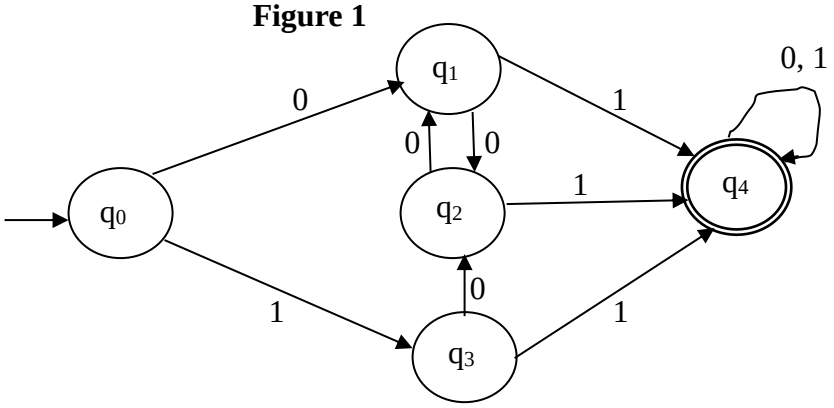
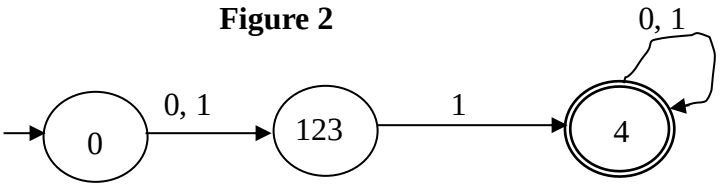



**Figure 2**




**2.4.2: Dead States**. A dead state is a state from which no accepting state can be reached. Algorithm 2.4 works under the following assumptions:

- The move function is total, that is there are transitions on all symbols from all states, or
- There are no dead states in the DFA.

To solve this problem, we have the following solutions

1) Make sure there are no dead states. This can be ensured by invariant or by explicitly removing dead states before minimisation.
2) Make sure there are no undefined transitions. This can be achieved by adding a new dead state (which has transitions to itself on all symbols) and replacing all undefined transitions by transitions to this dead state. After minimisation, the group that contains the dead state will contain all dead states from the original DFA. This group can now be removed from the minimal DFA (which will once more have undefined transitions)

**Further Reading:**

Try to understand the Subset construction Algorithm for the conversion of an NFA to a DFA outlined below.

**Algorithm:** Given an NFA N With states Q, $q_0 \in Q$, accepting states $F \subseteq Q$, transitions T and alphabet $\sum$, we construct an equivalent DFA D with states Q', starting state $q'_0$, accepting sates F' and a transition function move by:

$q'_0$ = λ- closure({q0})

move(Q' ,a) = λ- closure({t| q ∈ Q' and q$^c$t ∈ T})

Q' ={ $q'_0$} U {move(q', a) | q' ∈ Q', a ∈ ∑}

F' ={ q' ∈ Q' | q' n F ≠ {}}

The DFA uses the same alphabet as the NFA.

**NOTE Definition:** Given a set M of NFA states, we define λ- closure(M) to be the least (in terms of the subset relation) solution to the set equation

λ- closure(M) = M U {t|s ∈ λ- closure(M) and sct ∈ T}

Where T is the set of transitions in the NFA.